

SD Basic - Select Lists

Building, walking, saving and combining lists of record ids.

A select list is a list of record ids held by the session. Building one and reading it back is how a program processes a whole file, and how it picks up the result of a query run at the command line.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The eleven lists

There are **eleven** select lists, numbered `0` to `10`. List `0` is the default: every statement that takes a list number uses it when you do not say otherwise, and it is the one a query at the command line leaves behind.

A list is session state, not program state. It survives the program that made it, and a program that leaves one behind changes what the next command does. **Clear a list you built for your own use** — see `clearselect` below.

Building a list

```
select file.variable {to list} {on error statements}
sselect file.variable {to list}
selectn file.variable {to list}
selectv file.variable to list.variable
selectindex index.name {, value} from file.variable {to list}
formlist array {to list}
```

<code>select</code>	every id in the file, in no useful order
<code>sselect</code>	every id, sorted
<code>selectn</code>	as <code>select</code> , but does not clear the list first
<code>selectv</code>	the list into a variable rather than a numbered list
<code>selectindex</code>	from an alternate key index — see SD Basic - Alternate Key Indexes

`formlist`builds a list from a dynamic array you already have

On a file holding two records:

	RESULT
<code>select f</code> then a <code>readnext</code> loop	2 ids
<code>formlist 'X' : @fm : 'Y' : @fm : 'Z' to 7</code> then a loop	3 ids

`sselect` **sorts and** `select` **does not, and** `select` **is not in id order**. It returns ids in whatever order the file's hashing puts them, which changes as the file grows. **A report that must come out in order needs** `sselect`, or a query with a `by` clause; relying on `select` to be ordered produces output that is right on the developer's small test file and wrong in production.

Walking a list

```
readnext variable {from list} {on error statements}
  then statements
else statements

readnext variable, value.position, subvalue.position {from list}
```

`readnext` takes the next id and moves on. **The** `else` **branch is how the loop ends** — it fires when the list is exhausted.

```
select f
loop
  readnext id else exit
  read rec from f, id else continue
  ...
repeat
```

The exploded form — `readnext id, vpos, spos` — is for lists built from a multivalued index, where the same id appears once per matching value and the extra variables say which value matched.

A `readnext` **loop and a** `readu` **inside it is the commonest deadlock**. Two sessions walking the same file in different orders will each hold a record the other wants. Use `sselect` so both walk in the same order, and give the `readu` a `locked` clause so at least one of them gives up rather than waiting — see SD Basic - File Handling.

Asking about a list

```
selectinfo(list, key)
```

KEY	MEANING
1	is the list active?
3	how many entries it holds

After `select f to 3`, `selectinfo(3, 1)` is `1`; after `clearselect 3` it is `0`.

Key 3 is the only reliable way to count a file's records — `fileinfo()` has no record-count key. `select` the file and read `selectinfo(list, 3)`.

Clearing

```
clearselect {list}  
clearselect all
```

`clearselect` with no argument clears list 0. `clearselect all` clears all eleven.

Clear a list you no longer need, especially list 0: a program that returns to the command line leaving a list active makes the *next* command operate on that list instead of the whole file, which is a confusing thing to debug from the other end.

Saving a list

```
savelist name {from list} then ... else ...  
getlist name {to list} then ... else ...  
deletelist name
```

A saved list is a record in the account's `&SAVEDLISTS&` file and outlives the session.

A `savelist` of a two-id list followed by `getlist ... to 5` and a `readnext` loop on list 5 returned the same **2** ids.

`savelist` and `getlist` require a `then` or `else` clause, and the compiler's complaint does not say so.

Written without one, the error is "*Expected THEN or ELSE*" reported against whatever follows — which reads like a problem with the next statement. `else null` is enough:

```
savelist list.name from 3 else null getlist list.name to 5 else stop 'no such list'
```

`formlist` and `clearselect` do **not** take a `then`/`else` clause, so the four statements are not consistent with each other.

Combining and comparing lists

```
selectleft index.name from file.variable  
selectright index.name from file.variable  
setleft index.name from file.variable  
setright index.name from file.variable
```

These walk an alternate key index in either direction from the current position, rather than building a whole list at once.

`setleft` and `setright` position at the start or end without reading anything. They are covered with the rest of the index statements in SD Basic - Alternate Key Indexes.

```
listindex(list, delimiter, item)  
indices(file.variable {, index.name})
```

`listindex()` picks one entry out of a delimited list. `indices()` reports which alternate key indexes a file has, or the definition of one.

Lists as data

A list can be filtered in BASIC rather than by building a query string — which is usually clearer, and cannot be broken by a value containing a quote. `formlist` is the way back in.

```
select f to 2  
keep = ''  
loop  
  readnext id from 2 else exit  
  if id[1,3] = 'abc' then keep := id : @fm  
repeat
```

```
keep = trim(keep, @fm, 'T')          ;* see the warning below
formlist keep to 3
```

`selectv` **does not give you a dynamic array.** It puts the list into a variable of its own kind: `vartype()` of a `selectv` target is **11**, a select list. Concatenating it or calling `dcount()` on it fails with "*Data cannot be converted to a string*". **Read it with** `readnext variable from list.variable`, which returns the same three ids. `selectv` is for passing a list to a subroutine, not for inspecting one.

A trailing field mark becomes an extra, empty id. An array of three ids built with `keep := id : @fm` ends in a mark, and `formlist keep to 3` gives **four** entries — the fourth being the null string, which then reads a record that does not exist. **Strip the trailing mark**, or append the mark *before* each id except the first.

What is not here

Nothing in the select-list group has been removed from this port.

See also

SD Basic - File Handling · SD Basic - Alternate Key Indexes · SD Basic - Dynamic Arrays · SD Basic - Program Control.